Tallinn University

School of Digital Technologies

DTLGM.DT Digital Learning Games

Designing an XR Drone Training Simulator

Master's Thesis

Author: Jian Gong

Supervisor: Martin Sillaots

Tallinn 2026

Abstract

Automation is progressively displacing skilled technical labor, however, current robotic systems remain limited in replicating complex tasks, leaving a persistent need for human operators. At the same time, such skills are difficult to acquire, requiring prolonged training and the development of muscle memory. This creates a gap between the continued demand for skilled operators and the accessibility of training pathways.

This paper presents a design-based research study investigating Extended Reality (XR) as a medium for lowering the learning barrier and increasing engagement in complex skill acquisition, using First Person View (FPV) quadcopter piloting as a representative case domain.

This study is structured around three primary objectives: (1) the simulation of FPV quadcopter flight physics. (2) the design of a human AI collaboration framework (3) the interaction instruments for a high-fidelity XR prototyping. In particular PID and RL (Reinforcement Learning) are used in the co-training AI, and Geographic Information System (GIS) data integrated to support the environmental representation.

The contribution of this work is a structured design framework that links physical simulation, AI-assisted co-training, and XR-based interaction design, aimed at reducing the learning curve and improving accessibility to complex skill training.

Keywords: Extended Reality, FPV Drone, Quadcoptor, Quadrotor, Human AI collaboration, PID Control, Reinforcement Learning

Contents

Abstract **2**

Introduction **6**

Theoretical Background **8**

Introduction to Quadcopter Systems 8

Flight Dynamics and Principles of Operation 8

Pilot Interface and Control Systems 10

Proportional Integral and Derivative (PID Control) 13

Learning Theories in Simulation-Based XR 14

Experiential Learning in an XR environment 14

FPV Training Simulators 16

Human-AI Collaboration (HAC) 17

Reinforcement Learning (RL) 18

Human Computer Interaction in XR 19

Hardwares Introduction 19

Runtime and Standards 20

Spatial Coordinate System 22

Environment blend modes 22

Input Abstraction Layer 24

Methodology **26**

Design Methods and Development Strategy 26

Design-Based Research 26

Agile Development 26

Development Strategy 27

Comparison of Development Platforms 27

Simulate the Quadcopter . . . 29
Controller Mapping . . . 29
Flight Modes and Orientation Logic . . . 29
Design of the Human-AI Collaboration . . . 30
Design of a PID based HAC . . . 30
Design of a RL based HAC . . . 32
Comparison of the XR Interaction Frameworks . . . 34
Geographic Information Systems . . . 36
AI-assisted environment generation . . . 36
System Design Overview . . . 36
Archetecture . . . 36
Models . . . 36
Environment Construction . . . 36
Interaction and Control Design . . . 36
Learning System Design . . . 36
Data Collection Methods . . . 36

Results **37**
Control the Quadcopter in XR environment . . . 37
Path Following with PID Controller . . . 38
Hovering with RL Agent . . . 40
Tutorial on Game Interaction . . . 43
Control the Quadcopter in VR environment . . . 44
Drone Control Implementation . . . 44
Physics and Simulation System . . . 44
XR Interaction and UI Results . . . 44

Discussion **45**

RQ1: How Can FPV Quadcopter Flight Physics Be Simulated? 45
Comparison With Existing Simulators 45
RQ2: How can a human-AI collaboration agent be designed to assist users? 46
Comparison of PID and RL 46
RQ3: What interaction methods enable high-fidelity XR prototyping? 46
When XR helps 47
When XR is not preferable 47
Limitations 47
Self-Leveling Flight Mode Only 47
Visual Feedback Only 48
No Participant Testing 48
Future Work 48
Evaluation Methodology for XR-Based Drone Prototyping 48
Extending to Acrobatic Flight Mode 48
Designing Multimodal Feedback Mechanisms 49
Reinforcement Learning for Path-Following Control 49
Imitation Learning for Human-Guided Control 49

Summary **50**
Goals 50
Results 50
Generalization 50
Practical Use 50
Future Study 50

Appendix **56**

Introduction

Robots are increasingly capable of replacing routine manual and cognitive tasks. However, there remains a strong demand for human cognitive and motor skills that machines cannot yet replicate. At the same time, this technological shift is occurring alongside a growing labor shortage, partly because many workers are unwilling to take jobs that are physically demanding and are expected to be replaced by automation in the future.

As a result, industries such as agriculture, forestry, and heavy-duty mobile machinery are already facing significant shortages of skilled workers (Machado et al., 2021). Moreover, career expectations are increasingly influenced by perceptions of automation. Research shows that when individuals believe a job is likely to be replaced by artificial intelligence, their willingness to pursue that career decreases significantly (H. Wang et al., 2026).

Despite the growing level of automation, current robotic systems still struggle with complex and dynamic environments. Therefore, skilled human operators remain essential. However, acquiring these skills is difficult and time-consuming, as it requires extensive repeated practice to develop muscle memory.

In this context, learning theory provides a useful framework. Near transfer refers to applying skills in similar contexts, while far transfer refers to applying them in different contexts (Barnett & Ceci, 2002). For near transfer, repeated practice plays a critical role in building procedural knowledge and muscle memory.

Given these challenges, Extended Reality (XR) has emerged as a promising training approach. Previous studies have explored XR in domains such as medical procedures and aviation (Dutescu et al., 2025). Meanwhile, Unmanned Aerial Vehicles (UAVs) are widely used in construction, agriculture, logistics, and disaster management (Otto et al., 2018). Among these applications, First Person View (FPV) quadcopter piloting represents a particularly demanding skill that relies heavily on precise motor control and spatial awareness. Accordingly, this study uses FPV quadcopter piloting as a case to investigate XR-based training for muscle memory development.

However, existing drone simulators are primarily designed for entertainment rather than structured learning. Consequently, they often present steep learning curves and provide limited support for systematic skill acquisition. In addition, the lack of standardized controller configurations create further barriers (Ebeid et al., 2018), leading to user frustration even before effective training begins.

Taken together, low career expectations, high preparation effort, and early-stage frustration significantly reduce learner motivation. As a result, many beginners discontinue training before acquiring sufficient muscle memory.

To address these issues, the goal of this study is to design an XR-based training system that lowers the learning barrier for FPV quadcopter piloting. In particular, the proposed system aims to support muscle memory development through AI-assisted co-training and improved interaction design.

This study addresses the following research questions:

(1) How can FPV quadcopter flight physics be simulated?

(2) How can a human-AI collaboration agent be designed to assist users?

(3) What interaction methods enable high-fidelity XR prototyping?

This study uses a design-based research approach. It applies agile development practices and iterative prototyping to build an XR FPV quadcopter simulator, including physics simulation, AI co-training (PID control and reinforcement learning), and interaction design.

Theoretical Background

Introduction to Quadcopter Systems

A quadcopter, also frequently referred to as a quadrotor, is a type of multirotor helicopter or rotary-wing Unmanned Aerial Vehicle (UAV) that utilizes four rotors to achieve flight (Luukkonen, 2011). These rotors are typically directed upwards and arranged in a square formation, with each rotor positioned at an equal distance from the center of mass. In this configuration, the four actuators are generally equally spaced by 90° from one another (Idrissi et al., 2022).

The origins of quadrotor technology date back to 1907, when the Breguet brothers constructed the Gyroplane No. 1. While this marked one of the first manned helicopter flights, it was not a free flight; the machine lacked inherent stability and a proper control mechanism, requiring it to be tethered to the ground (Ghazbi et al., 2016).

The transition to modern UAVs occurred in the late 1990s. A pivotal moment was the 1999 release of the Draganflyer I, the first camera-equipped quadrotor to achieve mass production. This shifted quadrotor technology from a laboratory curiosity to a viable commercial product (Karahan, 2025).

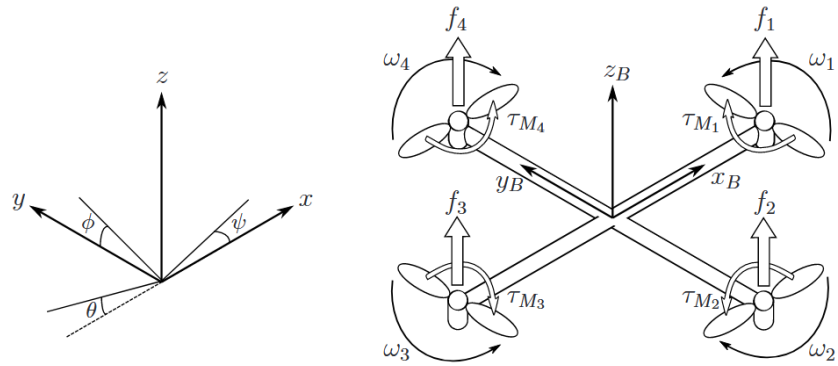
Since 2010, advancements in micro-electronics and control theory have expanded the use of quadrotors across various sectors:

- Public Safety: Search and rescue, firefighting, and border surveillance.
- Agriculture: Precision seeding, chemical spraying, and crop monitoring.
- Commercial: Last-mile delivery and high-end cinematography.

Flight Dynamics and Principles of Operation

A quadcopter is an under-actuated system, meaning it must manage six degrees of freedom (6DOF) as shown in Figure 1, three translational and three rotational—using only four control inputs (the angular velocities of the rotors).

Figure 1
The inertial and body frames of a quadcopter



The fundamental principle of quadcopter stability relies on torque cancellation. To maintain a stable orientation, two motors rotate clockwise (CW) while the adjacent pair rotates counter-clockwise (CCW). This arrangement ensures that the net moment about the center of mass is zero; without this pair-wise opposition, the drone would rotate uncontrollably around its vertical axis. Quadcopters typically use either a plus (+) or cross (x) configuration, with the cross configuration being more common due to its superior stability (Idrissi et al., 2022).

The absolute linear position of the quadcopter is defined in the inertial frame axes ($[x, y, z]^T$). The attitude, i.e.the angular position, is defined in the inertial frame with three Euler angles ($[\phi, \theta, \psi]^T$), which represent pitch angle, roll angle and yaw angle as shown in Figure 1.

Every change in position or attitude is achieved by varying the individual angular velocities (ω_i) of the four rotors:

Thrust (T): Simultaneously increasing all rotor speeds causes the vehicle to ascend. The force generated by rotor i is $f_i = k\omega_i^2$, where k is the lift constant. Total thrust is defined as show in Equation (1)

$$T = k \sum_{i=1}^{4} \omega_i^2 \tag{1}$$

Yaw (τ_ψ): Rotating the drone around the vertical axis is achieved by creating a torque imbalance between the CW and CCW pairs shown in Equation (2), where b is the drag constant.

$$\tau_\psi = b(\omega_1^2 - \omega_2^2 + \omega_3^2 - \omega_4^2) \tag{2}$$

Roll (τ_ϕ) and Pitch (τ_θ): These are achieved by creating a speed differential between opposite rotors. For a distance l from the center of mass, shown in Equation (3)

$$\begin{aligned} \tau_\phi &= lk(-\omega_2^2 + \omega_4^2) \\ \tau_\theta &= lk(-\omega_1^2 + \omega_3^2) \end{aligned} \tag{3}$$

The quadcopter is modeled as a rigid body using Newton-Euler equations. The acceleration in the inertial frame $[x, y, z]^T$ is governed by gravity (g) and the orientation of the thrust vector shown in Equation (4).

$$\begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} + \frac{T}{m} \begin{bmatrix} \cos\psi \sin\theta \cos\phi + \sin\psi \sin\phi \\ \sin\psi \sin\theta \cos\phi - \cos\psi \sin\phi \\ \cos\theta \cos\phi \end{bmatrix} \tag{4}$$

In the body-fixed frame, the angular acceleration is influenced by the external torque (τ), inertia matrix (I), and gyroscopic effects as shown in Equation (5)

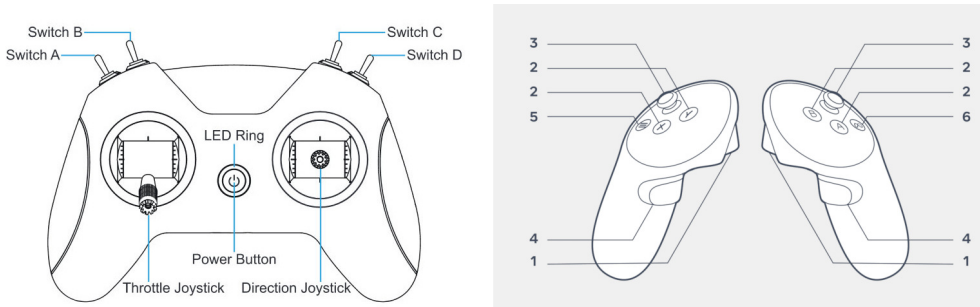
$$\begin{bmatrix} \dot{p} \\ \dot{q} \\ \dot{r} \end{bmatrix} = \begin{bmatrix} (I_{yy} - I_{zz})qr/I_{xx} \\ (I_{zz} - I_{xx})pr/I_{yy} \\ (I_{xx} - I_{yy})pq/I_{zz} \end{bmatrix} + \begin{bmatrix} \tau_\phi/I_{xx} \\ \tau_\theta/I_{yy} \\ \tau_\psi/I_{zz} \end{bmatrix} \tag{5}$$

Pilot Interface and Control Systems

The First-Person View (FPV) experience is defined by the synchronization between the pilot's inputs and the drone's visual feedback. An FPV drone can be controlled with a traditional remote controller or with a Motion Controller shown in Figure 2. A Motion Controller allows they user to maneuver the drone using hand gestures which is easy and convenient. The FPV Goggles is a also used for the FPV Drone. It's the defining factor for the FPV Drone. It connects to the drone's camera so that the camera user can have full vision of the quadcopter's camera

(Usama et al., 2021).

Figure 2
Transmitter (Left) and Gamepad (Right)



Adapted from (BetaFPV, 2022)

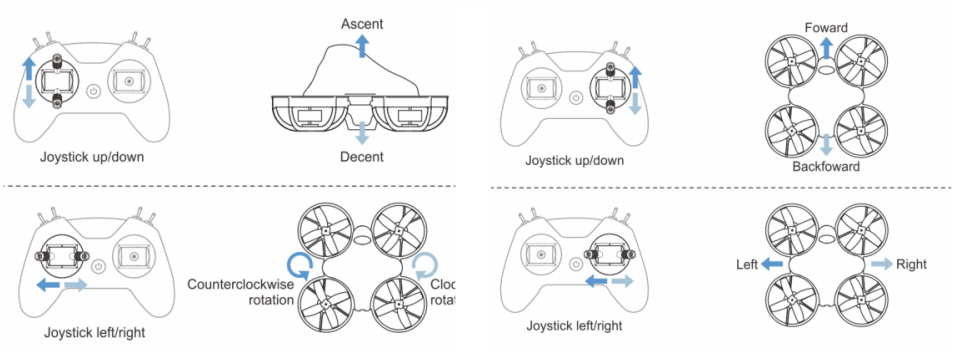
The industry standardizes these controls into two primary layouts: Mode 1 (Japanese Hand): Predominantly used in Japan and parts of Europe. The throttle is located on the right gimbal; Mode 2 (American Hand): The global standard for FPV.The throttle is located on the left gimbal, which most pilots find more intuitive for separating altitude from directional pitch. Difference between Mode 1 and Mode 2 is shown in Table 1

From a signal processing perspective, the flight controller differentiates between the unipolar throttle and the bipolar command sticks: Throttle (T): Mapped to a range of $[0,1]$. Roll, Pitch, and Yaw (ϕ, θ, ψ): Mapped to a range of $[-1,1]$. In Figure 3 shows the interaction of the transmitter in Mode 2 and how the quadcopter move.

Table 1
Comparison of Transmitter Control Modes

Mode	Left Stick (V)	Left Stick (H)	Right Stick (V)	Right Stick (H)
Mode 1	Pitch	Yaw	Throttle	Roll
Mode 2	Throttle	Yaw	Pitch	Roll

Figure 3
Mode 2 control system

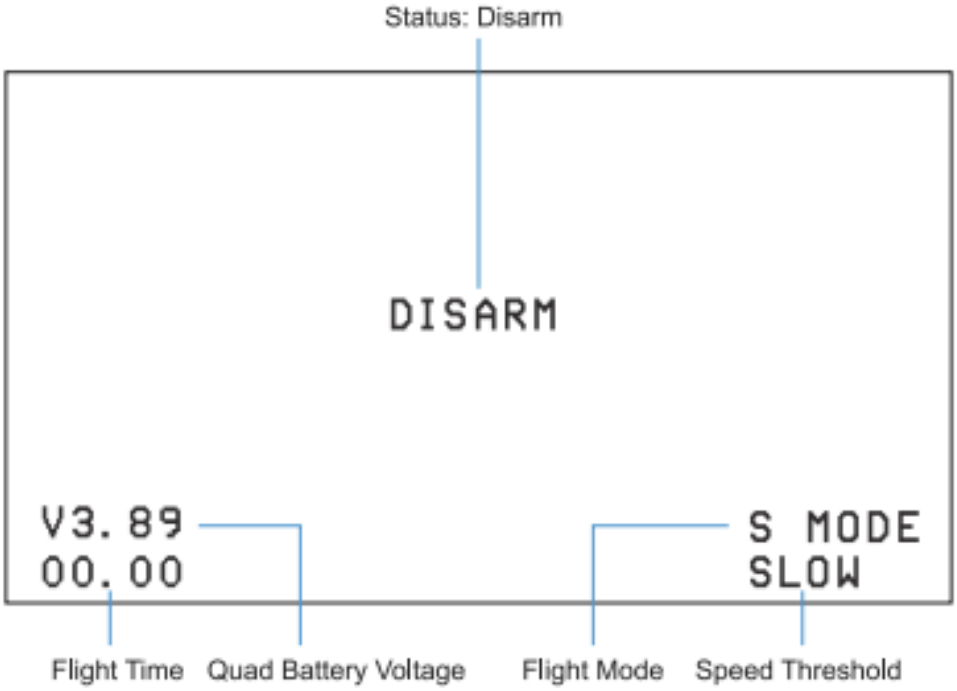


Adapted from (BetaFPV, 2022)

FPV drones generally operate across two flight logic modes. Self-Leveling (Angle/Normal Mode): The onboard flight controller uses accelerometers to automatically return the drone to a level horizon when the sticks are centered. This is ideal for beginners but limits maneuverability; Acrobat (Manual Mode): The controller only uses gyroscopes to maintain the current angular rate. If the pilot moves the stick and returns it to center, the drone maintains its current tilted orientation. This allows for flips and rolls but requires constant manual correction.

The FPV goggles provide the pilot with a low-latency first person view video feed pass from the camera set on the quadcopter's front. Beyond the video feed, an On-Screen Display (OSD) provides critical telemetry data, including battery voltage, signal strength, and flight time, as shown in Figure 4.

Figure 4
The FPV goggles interface



Adapted from (BetaFPV, 2022)

Proportional Integral and Derivative (PID Control)

In 1922, Nicolas Minorsky presented a very clear analysis of the control actions necessary to provide effective control of a system whose exact dynamics were un- known. He analyzed the actions taken by a good helmsman steering a ship and translated these actions into the appropriate mathematical formulations (Bennett, 1993). It is now know as the Proportional-Integral-Derivative (PID) controller. PID is a type of controller that is most commonly used and applied in mechanical systems (Idrissi et al., 2022). Also used in many modern quadcopters, PID is employed here to stabilize drone movement and to function as a trainer AI.

The controller consists of three main components: Proportional (K_p): Acts like a virtual spring that pulls the system toward the desired position; the control action increases as the error gets larger; Integral (K_i): Accumulates the error over time to eliminate steady-state error, ensuring

the drone reaches the exact target even in the presence of disturbances like gravity; Derivative (K_d): Acts as a virtual damper to reduce oscillations and prevent the system from overshooting its target.

The idealized control law is expressed as Equation (6).

$$u(t) = K_p e(t) + K_i \int e(t)dt + K_d \frac{de(t)}{dt} \tag{6}$$

Learning Theories in Simulation-Based XR

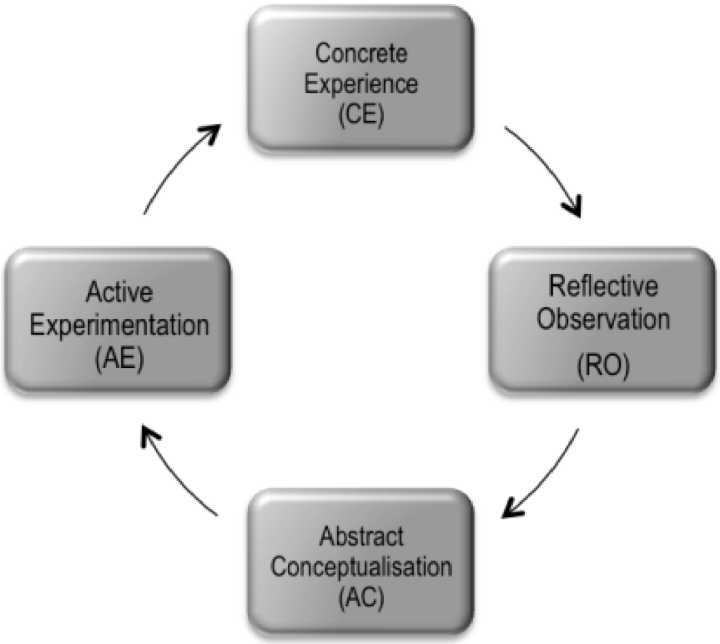
Experiential Learning in an XR environment

Experiential learning theory posits that learning is a process through which knowledge is created via the transformation of experience (Kolb, 1984). This perspective emphasizes that learning emerges through the integration of experiencing, reflecting, thinking, and acting within a continuous cycle.

Kolb conceptualizes learning as a four-stage cyclical process consisting of concrete experience (CE), reflective observation (RO), abstract conceptualization (AC), and active experimentation (AE) (Almeida & Mendes, 2010), as illustrated in Figure 5.

Concrete experience refers to the learner's direct engagement in a specific activity, where immersive technologies such as extended reality (XR) can enhance the sense of presence. Reflective observation involves consciously reflecting on the experience, often supported by feedback mechanisms or guidance;in this context, a human-AI collaboration agent can provide timely and accurate support to help learners interpret their experiences. Abstract conceptualization entails the formation or refinement of theoretical constructs and hypotheses derived from reflection. Finally, active experimentation involves applying these conceptualizations to new situations, allowing learners to test their understanding and generate new experiences, thereby continuing the learning cycle.

Figure 5
The Four-Stage Learning Cycle by Kolb (1984)



Prior research suggests that virtual reality (VR), as an experiential learning tool, can effectively enhance technical skills and learner confidence, even with relatively short exposure times (Knobovitch et al., 2025). VR also enables personalized learning experiences by allowing students to explore virtual environments at their own pace and according to their individual preferences. Furthermore, VR systems can support improved comprehension through the delivery of adaptive and personalized feedback (Marougkas et al., 2023).

In addition, the immersive and interactive nature of XR has been shown to promote higher levels of engagement, both physically and cognitively (Liu et al., 2025). Compared with flat display devices, XR-based approaches can also improve learner concentration. For example, Liu et al. (2025) found that learners in a VR-based instruction (VRI) group demonstrated higher task mastery rates than those in other groups. This improvement was attributed to increased engagement and interactivity, which supported better information retention.

FPV Training Simulators

Training Simulators are designed to imitate the operations and behaviors of a facility or process, so as to develop the skills or experiences of the operators who use them. The objective of the operator of the Training Simulator is to maximize his performance in achieving the task being simulated.(Narayanasamy et al., 2006). Simulation-based learning as an active and immersive experience in a virtual learning environment that recreate a realistic and authentic real-life event or a set of learning situations to solve a problem.(Dai & Ke, 2022)

In aviation and drone domains, such systems replicate flight dynamics, cockpit instrumentation, and environmental conditions of real aircraft. Flight training simulators have a long history, beginning around 1910 and evolving significantly during the World Wars. By the 1960s, they had become essential tools for pilot training, enabling the safe practice of complex scenarios. With advances in artificial intelligence and related technologies, modern simulators can now provide real-time feedback based on expert pilot performance. Today, they are widely used in both civilian and military contexts for training and mission planning (Pappas et al., 2025).

An FPV (First-Person View) training simulator is a specialized computer-based training system designed to teach and refine the technical skills required to operate FPV drones. Similar to traditional flight simulators, FPV simulators are widely used across entertainment, professional training, and research contexts.

Existing literature on drone simulators has primarily focused on technical design or training effectiveness, often using research-oriented platforms such as Microsoft AirSim (Shah et al., 2017). However, widely used commercial FPV simulators (e.g., Liftoff and Uncrashed) remain under explored in pedagogical research. These commercial simulators typically emphasize high-fidelity environments and support advanced flight modes such as acrobatic mode for pilot practice. In contrast, research platforms such as AirSim and RotorS are widely used for machine learning and autonomous system development, but are not specifically tailored for practical pilot training (Pappas et al., 2025).

Despite their realism, FPV simulators also present several limitations. For example, while

they aim to replicate real-world physics, users often report a steep learning curve and highly unforgiving flight dynamics, which may not always support beginner training effectively (Guide, 2026). Additionally, hardware compatibility issues can arise, as not all radio transmitters are universally supported. The lack of standardization in flight controller architectures, along with the prevalence of proprietary and closed-source systems in UAV platforms, further complicates integration and usability (Ebeid et al., 2018).

Human-AI Collaboration (HAC)

Game AI, also referred to as 'AI in games', is a broad research field that studies the application of artificial intelligence within and for games (García-Sánchez, 2019). It is widely used in both board games and video games.

Dablander (2024) identify three major application domains of game AI: (1) non-player character (NPC) control and agent modeling, (2) procedural content generation, and (3) player modeling. More specifically, AI for game playing and agent modeling includes simulating human players or controlling in-game agents such as NPCs or environmental systems. AI-based procedural content generation focuses on the algorithmic creation of game elements, including levels, music, textures, dialogue, items, and characters. In addition, player modeling aims to infer player characteristics such as play style, behavioral patterns, or emotional states based on gameplay data.

In the intelligent era, the interaction between humans and intelligent systems fundamentally involves collaboration with autonomous intelligent agents. Human-AI Collaboration (HAC) represents a novel type of human-machine relationship facilitated by autonomous intelligent machines equipped with AI technologies. In this paradigm, AI agents serve not only as auxiliary tools but also as active teammates, partnering with humans to accomplish tasks collaboratively.(Gao et al., 2025)

One prominent form of HAC is mixed-initiative co-creativity (MI-CC), in which both humans and AI systems actively participate in the creative process. MI-CC systems have been applied in domains such as game design, programming (e.g., GitHub Copilot), visual art

generation, and storytelling. Although MI-CC systems have been widely developed, research suggests that human factors underlying effective collaboration with creative AI remain under explored compared to system development itself (Lin et al., 2023).

Beyond creative domains, HAC is also increasingly adopted in practical applications such as software testing. For instance, AI-assisted approaches have been proposed to complement or replace traditional manual game testing workflows, improving efficiency and coverage (Zhang et al., 2025).

Furthermore, HAC has been investigated in safety-critical domains such as autonomous driving, where human-AI collaboration is used to support decision-making and enhance system robustness. These studies highlight the potential for synergistic interaction between human operators and AI agents across diverse application areas (Gao et al., 2025).

In the context of this study, an AI agent designed for drone piloting can provide reflective feedback to the learner. Such feedback supports cognitive reflection and facilitates the formation of abstract conceptualization, which corresponds to the second stage of Kolb's experiential learning cycle (Kolb, 1984).

Reinforcement Learning (RL)

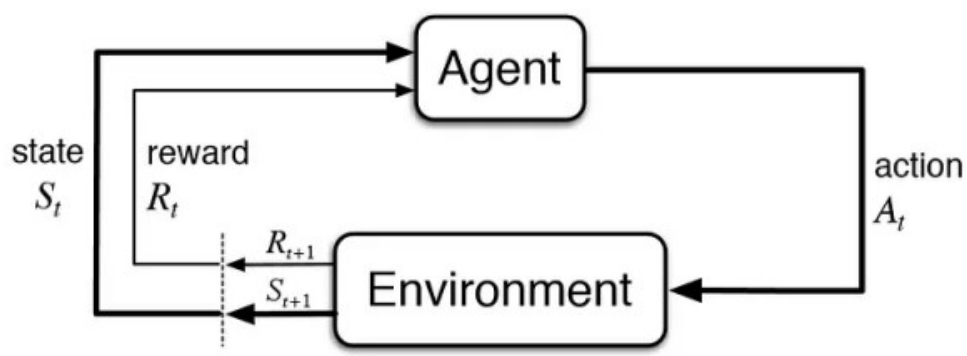
Proportional-Integral-Derivative (PID) control is widely used in control systems due to its simplicity. However, as a model-based (white-box) approach, it may exhibit limitations in highly nonlinear or complex environments, where tuning the integral component alone is often insufficient for optimal performance. In contrast, Reinforcement Learning (RL) has emerged as a data-driven (black-box) alternative for handling such complex control problems. To reduce the computational burden on onboard systems, reinforcement learning has increasingly been adopted as a key approach for autonomous obstacle avoidance (Yu et al., 2025).

The conceptual foundation of RL can be traced back to early work in artificial intelligence. Turing (1948) proposed early ideas of reward and punishment mechanisms in learning systems (Barto, 2019). Modern reinforcement learning formalizes this idea as a computational framework in which an agent learns optimal behavior through trial-and-error

interactions with an environment (Kaelbling et al., 1996). Unlike supervised learning, RL does not rely on labeled input-output pairs. Instead, the agent receives scalar reward signals and aims to maximize cumulative long-term reward.

The mathematical foundation of RL is typically formulated using Markov Decision Processes (MDPs). In an MDP, the agent's actions influence both immediate rewards and future environmental states. As illustrated in Figure 6, an MDP is formally defined by four components: (1) a set of states S, representing all possible configurations of the environment; (2) a set of actions A, representing the decision space of the agent; (3) a reward function $R : S \times A \to \mathbb{R}$, which defines the immediate reward received after taking an action in a given state; and (4) a state transition function $T : S \times A \to \Delta(S)$, which defines the probability distribution over next states s' given the current state s and action a.

Figure 6
Markov Decision Process diagram



In the context of training simulators, reinforcement learning provides a promising approach for developing autonomous or semi-autonomous agent behaviors that can support adaptive training environments and human-AI collaboration systems.

Human Computer Interaction in XR

Hardwares Introduction

Modern extended reality (XR) hardware can be classified based on the location of its compute unit into two categories: PC-tethered virtual reality (PCVR) systems and standalone

(all-in-one) systems.

PCVR (Tethered). The headset acts as a peripheral. Assets are rendered on a high-end PC and streamed to the device via a wired (Link) or wireless (Air Link) connection. This allows developers to utilize the full power of engines like Unreal Engine, enabling high-fidelity features such as Lumen and Nanite. PC VR also supports specialized peripherals like haptic suits or omnidirectional treadmills, as seen in industry-standard titles like Half-Life: Alyx.

Standalone (All-in-One). These headsets are self-contained systems with internal mobile chipsets. These provide high mobility but require significant optimization. High-end features must often be disabled or simplified to maintain performance. However, lightweight engines like Unity or Godot excel in this space, offering a balance between visual quality and mobile performance.

(Rendevski et al., 2022) note that Standalone VR (SVR) applications offer significant advantages over PC VR. With proper optimization techniques, almost any PC VR scenario can be developed and deployed as SVR while still providing an acceptable user experience. Differently, a recent survey from (Game Developers Conference, 2026) shows that among developers interested in creating VR/AR/MR games, the SteamVR platform (PC VR) slightly surpassed the Meta Quest / Horizon Store (68% vs. 64%) as their platform of interest. These were followed by PlayStation VR/VR2 (34%) and Apple visionOS (15%). However, when filtering for developers who have actually worked on VR/AR/MR games, a strong preference remains for the Meta Quest (72%) over SteamVR (59%).

Due to hardware availability, this paper focuses exclusively on development for the Meta Quest 3.For Head-Mounted Devices (HMDs) like the Meta Quest, two primary architectural configurations exist:

Runtime and Standards

To ensure cross-platform compatibility, developers rely on unified industry standards. Currently, there are two major public standards, OpenXR and WebXR.

OpenXR. This is the industry-standard API for XR applications. It is the interface between an application and an in-process or out-of-process XR runtime system. The runtime system handles functionality such as frame composition, peripheral management, and raw tracking information (Khronos OpenXR Working Group, 2024). Major game engines and the Meta Spatial SDK are built upon OpenXR to ensure hardware-agnostic development.

A primary bottleneck in XR development is the deployment loop. Compiling a standalone APK and pushing it to a device can take 10 15 minutes per iteration. There are two solutions to avoid the compiling: (1) Link Mode: Developers often use the Oculus Link App to preview scenes in PC VR, though this requires constantly putting on and taking off the headset. (2) Simulators: The Meta XR Simulator is increasingly becoming the preferred method for rapid iteration, allowing developers to simulate head and controller movements directly on their PC without needing physical hardware.

WebXR. The Device API provides the interfaces necessary to enable developers to build compelling, comfortable, and safe immersive applications on the web across a wide variety of hardware form factors (World Wide Web Consortium, 2022).

WebXR serves as a higher-level abstraction that runs in the browser, combining older specifications from WebVR and WebAR. The WebXR standard shares core features with and can be viewed as a subset of OpenXR. While traditional game engines offer WebXR export support, there are fewer dedicated development stacks built exclusively for it.

For security reasons, WebXR APIs are restricted to HTTPS.Consequently, developers must use port-forwarding technologies during local development. Thanks to the efficiency of the web stack, a WebXR application typically only requires a few seconds to bundle. It is worth noting, however, that web rendering technology is currently transitioning from WebGL2 to WebGPU, which may introduce API changes in the WebXR ecosystem.

Although web technologies are widely used across various domains, WebXR remains an emerging technology and is still considered relatively unstable. Therefore, this study adopts the OpenXR standard to ensure greater stability and cross-platform compatibility.

Spatial Coordinate System

In an XR environment, a coordinate system is used as a frame of reference (FoR) to determine the position, orientation, and scale of virtual objects relative to the surrounding environment (Davari & Bowman, 2024).

Across both virtual reality (VR) and augmented reality (AR), XR applications require consistent mapping between virtual objects and their corresponding real-world locations. To support this, OpenXR defines spatial constructs known as reference spaces, which enable applications to track the real world and describe how different frames of reference relate and evolve over time (Khronos OpenXR Working Group, 2024).

OpenXR specifies four primary reference spaces: VIEW, LOCAL, STAGE, and LOCAL_FLOOR.

The VIEW space represents the pose of the primary viewer and is typically used for head-locked content, such as heads-up displays (HUDs). The LOCAL space provides a stable, gravity-aligned coordinate system suitable for seated or stationary experiences. The STAGE space defines a room-scale coordinate system aligned with the physical floor, enabling spatial movement within a bounded area. Finally, the LOCAL_FLOOR space is similar to LOCAL but with its origin explicitly positioned at an estimated floor level, ensuring consistency with real-world ground alignment.

Environment blend modes

In XR systems, rendered images must be composited with the system display to produce the final user-perceived output. According to Khronos OpenXR Working Group (2024), this process is governed by one of the three environment blend modes: opaque, additive, and alpha blend.

The term environment in this context refers either to the user's perception of the physical world beyond the display or to a fully synthetic environment generated by the system. In many XR applications, it may also represent a combination of both real and virtual elements. Environment blend modes therefore define how virtual content is integrated with, or replaces, the

real-world view.

Opaque blending fully replaces the real-world view with virtual imagery and is typically used in virtual reality (VR), where users are immersed in a completely computer-generated environment. In contrast, augmented reality (AR) aims to overlay digital content onto the physical world, aligning virtual elements spatially and perceptually with the real environment (Schmalstieg & Höllerer, 2016).

Two primary display technologies are commonly used in AR systems: optical see-through (OST) and video see-through (VST). In optical see-through head-mounted displays (HMDs), users view the real world directly through semi-transparent mirrors, onto which virtual imagery is projected. In video see-through systems, the real-world view is captured by cameras and digitally composited with virtual content before being displayed to the user (Billinghurst et al., 2015; Rolland & Hua, 2005).

These display technologies correspond closely to different blending strategies. Additive blending is typically associated with optical see-through displays, where virtual content is added to the incoming real-world light without fully occluding it. Alpha blending, on the other hand, is commonly used in video see-through systems, where pixel-wise compositing allows for proper occlusion and depth integration between real and virtual elements.

Recent developments in HMDs increasingly rely on video passthrough technology. In such systems, users perceive the physical environment indirectly through stereoscopic, real-time video feeds displayed inside the headset rather than through direct optical transmission (Bailenson et al., 2024). This approach enables more flexible and accurate compositing of virtual and real-world elements.

Empirical studies have shown that passthrough-based approaches can enhance user experience. For instance, Waldow et al. (2025) introduced the concept of Pass-Through Embodiment (PTE) and demonstrated that allowing users to see their own physical bodies in real time significantly improves presence and embodiment without negatively affecting task performance or inducing simulator sickness.

Overall, the choice of environment blend mode and display technology plays a critical role in shaping the perceptual and interaction characteristics of XR applications. Leveraging modern passthrough capabilities can enable more immersive and context-aware mixed reality experiences, which is particularly valuable in application domains such as simulation and training.

Input Abstraction Layer

In OpenXR, an interaction profile path defines a collection of input components (e.g., buttons, triggers, and sensors) arranged in a standardized physical configuration. This abstraction enables applications and runtime systems to coordinate action bindings independently of specific hardware implementations.

Interaction profiles generalize various input controllers, hand gestures, and other tracking systems into a unified set of action types XrActionType. These include boolean inputs, scalar values (float), two-dimensional vectors (vector2f), and pose data, which represent position and orientation in two- or three-dimensional space (Khronos OpenXR Working Group, 2024).

The hardware layout of the Meta Quest 3 Touch Plus controllers is illustrated in Figure 2, while the functional mapping of each input component is summarized in Table 2.

Table 2
Hardware Layout of Meta Quest 3 Touch Plus Controllers

Component	Description
Index Trigger (1)	Analog trigger located on the front, typically used for selection or firing actions.
Face Buttons (2)	Left controller: X and Y buttons; right controller: A and B buttons.
Thumbstick (3)	Analog stick used for navigation or movement; includes a clickable button (L3/R3).
Grip Button (4)	Analog side button commonly used for grabbing interactions.
Menu Button (5)	Located on the left controller; used to open in-application menus.
Meta Button (6)	Located on the right controller; used to access the system-level menu.
Thumbrest	Capacitive surface for detecting thumb placement.

Raycasting is a widely used pointing-based interaction technique in extended reality (XR), where a virtual ray is projected from an input device (e.g., a controller) into the virtual environment to enable distant object selection. The ray is typically defined by the pose of the controller, with its origin at the device position and its direction aligned with its forward

orientation (Bowman & Hodges, 1997; Mine, 1995).

In modern XR systems such as OpenXR, raycasting is commonly implemented using pose-based input actions. Combined with discrete inputs such as the index trigger or grip button, users can effectively select and manipulate objects within a three-dimensional environment.

Methodology

Design Methods and Development Strategy

Design-Based Research

Design-based research (DBR) is a research methodology aimed at improving educational practices through systematic, flexible, and iterative processes of analysis, design, development, implementation, and evaluation. It is grounded in collaboration between researchers and practitioners in real-world settings, ultimately producing design principles or theoretical contributions (H. Wang et al., 2026).

DBR is primarily conducted in authentic educational environments to bridge the gap between theoretical research and practical application (Jong et al., 2017). Unlike traditional laboratory-based approaches, DBR is situated in naturalistic contexts where social interactions and environmental complexity are preserved and considered as integral parts of the research process (H. Wang et al., 2026).

A central feature of DBR is its iterative cycle, which consists of design, enactment, analysis, and redesign. The outputs of each cycle inform and guide subsequent cycles, enabling continuous refinement of both the intervention and the underlying theory (Jong et al., 2017).

Agile Development

Beck and Andres (2004) define Agile as a set of values and principles that emphasize iterative development, customer collaboration, and responsiveness to change. Agile development approaches typically include iterative and incremental development, self-organizing teams, and continuous feedback and adaptation.

Agile processes focus on the rapid and early delivery of working software and are based on development models that support incremental and iterative progress (Keith, 2010).

A user story is a description of a feature expressed from the perspective of the user and its intended value. User stories serve as the core unit of work in Agile development. Multiple user stories are maintained in a backlog, from which a subset is selected for implementation during a time-boxed iteration, typically lasting two to four weeks, known as a sprint. A collection of

related user stories that collectively deliver a larger functional capability is referred to as an epic. Completed sets of functionality, such as alpha or beta versions, are commonly referred to as milestones.

User stories must be estimated, which requires sufficient understanding of both the system being built and the implementation approach. If the scope of a story is too large or insufficiently understood, accurate estimation becomes difficult. Such cases are often addressed through spikes, which are time-boxed research tasks used to reduce uncertainty (Keith, 2010).

Both DBR and Agile share a common iterative philosophy, in which development proceeds through cycles of analysis, implementation, evaluation, and refinement, enabling continuous learning and adaptation throughout the process.

Development Strategy

This study includes three objectives in development, (1) Choosing or implement an FPV quadcopter physics; (2) Implement and testing a human-AI collaboration agent; (3) Choosing a set of interaction UI components and implement the XR environment.

Comparison of Development Platforms

Game development is strongly influenced by the choice of game engine. Before initiating a project, it is critical to determine the target hardware and environmental requirements, as these factors define the development constraints.

This section presents a comparison of current game engines supporting extended reality, based on programming language, physics engine, rendering methods, Meta platform support, machine learning support, and editor size (see Table 3). This comparison is based on official documentation and prior research from (Ezhil, 2025).

Unreal Engine: Unreal Engine is widely recognized for its photorealistic rendering capabilities and high visual fidelity. Its Chaos Physics engine is highly regarded for complex simulations, making it the foundation for projects like Microsoft's AirSim. However, UE has a high barrier to entry: the editor is massive, and utilizing Meta's private APIs often requires compiling a custom engine branch from source.

Table 3
Software Engine Comparison

Feature	Unreal Engine (5.7)	Unity (6.0)	Godot (4.6)	Wonderland Engine
Language	C++, Blueprints	C#	GDScript, C#	JavaScript, TypeScript
Physics	Chaos Physics	NVIDIA PhysX, DOTS	Jolt Physics	NVIDIA PhysX
Rendering	Lumen, Nanite	URP, HDRP	Forward+, Mobile	WebGL2
Meta Support	Fork build	First-class	Community-driven	No support
ML Support	Learning Agent	ML Agents	Godot RL Agents	No support
Editor Size	∼50GB+	∼20GB	$<$ 1GB	200MB

Unity: Unity remains the most popular choice for Meta Quest development due to its mature SDK support and C# scripting. While it uses PhysX by default, its DOTS (Data-Oriented Technology Stack) provides high-performance physics for complex drone swarms. It offers the most streamlined out-of-the-box experience for Standalone VR.

Godot: Godot has emerged as a powerful, lightweight alternative. It is highly compatible with modern AI coding assistants due to its Python-like GDScript. The integration of Jolt Physics as the default 3D engine has resolved previous performance hurdles for VR. However, its XR ecosystem is largely community-maintained. While excellent for prototyping, the lack of official Meta API support and the potential instability of third-party GDExtensions require careful dependency management for production-grade projects.

Wonderland Engine: Wonderland Engine is a specialized WebXR engine built in C++ and compiled to WebAssembly. By leveraging WebGL2, it achieves the performance necessary to render thousands of objects at native frame rates directly within a browser. The development workflow relies on JavaScript and TypeScript, providing a highly familiar and accessible environment for traditional web developers. A unique advantage is the ability to edit scenes directly on the headset, enabling instant feedback without removing the device.

Unity 3D is frequently used in research due to its flexible configuration, extensive plugin

ecosystem, and ease of rapid prototyping. Thus, it is often used in various experiments (Shatokhin et al., 2025).

Simulate the Quadcopter

This section examines the flight dynamics and control implementation found in the Yue's Ultimate Drone physics model (Yuetility-Studios, 2022).

Controller Mapping

The flight controller distinguishes between unipolar throttle inputs and bipolar command sticks (Roll, Pitch, and Yaw): Throttle (T): Mapped to a normalized range of $[0,1]$. Attitude Commands (θ, ϕ, ψ): Mapped to a range of $[-1,1]$.

To refine the pilot's control over the aircraft, the system utilizes a non-linear control curve for joystick calculation. As (Lovesay, 2003) observes, a good quality force tracking controller has a linear output, with joystick force directly proportional to sight velocity, but many systems shape the joystick voltage output, to achieve fine control and greater tracking accuracy as the joystick output approaches zero, whilst retaining the same maximum output capability.

In this implementation, shaping is achieved by combining proportional (p) and exponential (e) components to calculate the target angular rate (τ) from the raw angular rate (u) as shown in Equation (7).

$$\tau = p \cdot u + e \cdot \operatorname{sgn}(u) \cdot u^2 \tag{7}$$

The raw gamepad throttle $u_T \in [-1,1]$ is normalized to a linear thrust scalar as shown in Equation (8)

$$T = \frac{u_T + 1}{2} \tag{8}$$

Flight Modes and Orientation Logic

Since the Unity engine utilizes the FixedUpdate method for physics calculations, the drone's target orientation is updated discretely at every physics timestep (Δt).

In Acro mode, the joystick inputs command the angular rate of change. The drone's target orientation q_{target} is an accumulation of these rates over time. The orientation is updated by integrating the processed angular rates $(\tau_\phi, \tau_\theta, \tau_\psi)$ as shown in Equation (9), the Quaternion.Euler equation is Equation (5).

$$\begin{gathered} q_\Delta = \text{Quaternion.Euler}(\tau_\theta \Delta t, \tau_\phi \Delta t, \tau_\psi \Delta t) \\ q_{\text{target},k} = q_{\text{target},k-1} \cdot q_\Delta \end{gathered} \tag{9}$$

In Self-Leveling mode, intended to prevent crashing and ease navigation, the pitch and roll sticks command a specific angle rather than a rate. Given a predefined maximum bank angle for pitch (θ_{max}) and roll (ϕ_{max}), the target values are calculated as Equation (10).

$$\theta_{\text{target}} = u_\theta \cdot \theta_{\text{max}}, \quad \phi_{\text{target}} = u_\phi \cdot \phi_{\text{max}} \tag{10}$$

The simulation applies two primary vectors to the drone's Rigidbody: appliedForce (Thrust) and appliedTorque (Attitude Control).

The total thrust force $\mathbf{F}_{\text{thrust}}$ is applied along the vehicle's local up vector ($\hat{\mathbf{u}}_{\text{up}}$). The magnitude is governed by the throttle T and the maximum thrust constant F_{max}, the thrust force is calculated as shown in Equation (11).

$$\mathbf{F}_{\text{thrust}} = (T \cdot F_{\text{max}}) \cdot \hat{\mathbf{u}}_{\text{up}} \tag{11}$$

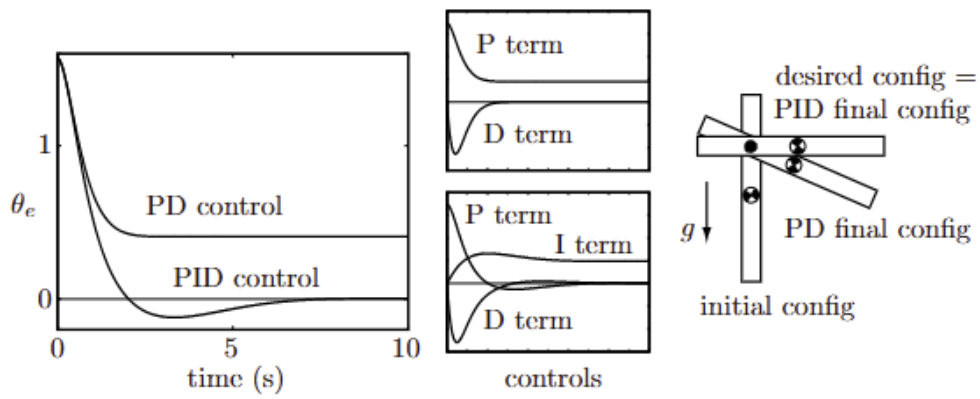
Design of the Human-AI Collaboration

Design of a PID based HAC

Proportional integral derivative (PID) shown in Equation (6) can be applied for making a simple aviation helper in piloting the quadcopter. For basic flight stabilization, a simple PD action can provide satisfactory performance shown in Figure 7.

For attitude control, The error between the target orientation q_{target} and the current body orientation q_{body} is defined as Equation (12).

Figure 7
Comparison of PID and PD



$$\Delta q = q_{\text{target}} \cdot q_{\text{body}}^{-1} \tag{12}$$

Using Unity's ToAngleAxis method, this relative rotation is converted into an angle ϕ and an axis $\mathbf{n}$. The error vector $\mathbf{e}$ is then $\mathbf{e} = \phi \cdot \mathbf{n}$.

The controller applies a PD loop (Proportional-Derivative) to generate corrective torque shown in Equation (13).

$$\tau_{\text{world}} = K_p \cdot \mathbf{e} + K_d \cdot \frac{\mathbf{e} - \mathbf{e}_{k-1}}{\Delta t} \tag{13}$$

The world-space torque is transformed into the body frame (R_{wb}) for the physics simulation: $\tau_{\text{body}} = R_{\text{wb}} \cdot \tau_{\text{world}}$

The controller should be able to pilot the quadcopter in a closed-loop mission. The system employs a cascaded control scheme where the Outer Loop (Position) provides setpoints for the Inner Loop (Attitude/Velocity).

The Inner Loop features controllers that yield the required rolling and pitching moments τ_x and τ_y to regulate horizontal movement. The Inner Loop block requires the current quadcopter attitude as well as reference signals for roll and pitch to obtain its output (Paredes et al., 2021); The Outer Loop, which features controllers that yield the required thrust differential and yawing

moments as well as the desired reference signals.

During takeoff, the controller focuses exclusively on vertical error (e_y). The vertical force $forceY$ is mapped to a normalized throttle value (T) shown in Equation (8).

Following a path requires a look-ahead method. Given current position **p** and look-ahead target **t** on the path, the position error is defined as: $e_x = t_x - p_x, \quad e_y = t_y - p_y, \quad e_z = t_z - p_z$.

The Outer Loop converts this error into a desired velocity in the world axis as shown in Equation (14).

$$v_{x,\text{des}} = \text{PD}_X(e_x), \quad v_{z,\text{des}} = \text{PD}_Z(e_z) \tag{14}$$

To translate these into drone-relative commands (Pitch/Roll), the world velocity is projected onto the drone's Heading Axis: (1) Forward Basis: $\hat{\mathbf{f}} = R_y(\psi)\,\hat{\mathbf{z}}_{\text{forward}}$; (2) Right Basis: $\hat{\mathbf{r}} = R_y(\psi)\,\hat{\mathbf{x}}_{\text{right}}$; The projections shown as Equation (15). Where $e_{v,\|}$ is the longitudinal velocity error, $e_{v,\perp}$ is the lateral velocity error.

$$v^{\|}_{\text{des}} = \mathbf{v}_{H,\text{des}} \cdot \hat{\mathbf{f}}, \quad v^{\perp}_{\text{des}} = \mathbf{v}_{H,\text{des}} \cdot \hat{\mathbf{r}} \tag{15}$$

The Inner Loop then calculates the error between desired and actual velocity to produce stick inputs. Pitch (Right Vertical): Based on $e_{v,\|}$; Roll (Right Horizontal): Based on $e_{v,\perp}$.

The target yaw speed $\psi_{\text{target}} = \text{atan2}(v_x, v_z)$, the yaw error is $e_\psi = \text{DeltaAngle}(\psi, \psi_{\text{target}})$, so mapped to joystick movement is $u_{\text{yaw}} = \text{PID}_{\text{yaw}}(e_\psi)$. The other axis are the same methods.

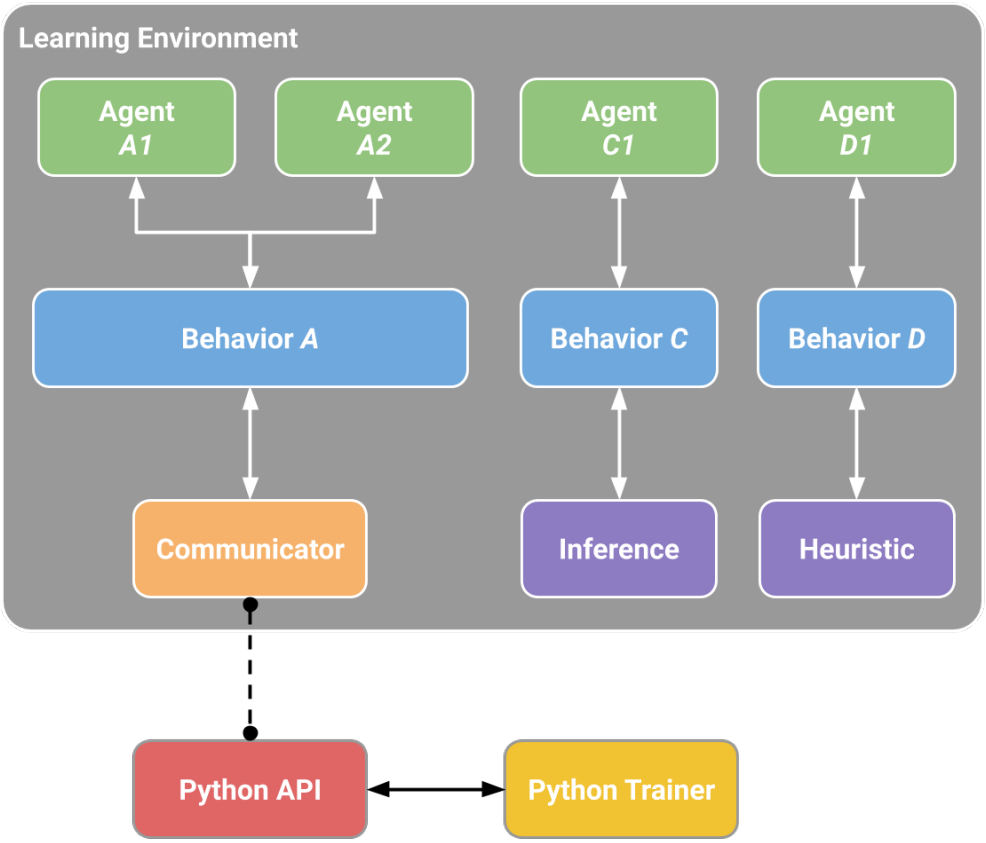
Design of a RL based HAC

Unity's ML-Agents Figure 8 enable games and simulations to serve as environments for training intelligent agents in Unity. Training can be done with reinforcement learning, imitation learning, neuroevolution, or any other methods. Trained agents can be used for many use cases, including controlling NPC behavior (in a variety of settings such as multi-agent and adversarial), automated testing of game builds and evaluating different game design decisions pre-release.

Every Learning Environment will always have one Agent for every character in the scene.

While each Agent must be linked to a Behavior, it is possible for Agents that have similar observations and actions to have the same Behavior.

Figure 8
Simplified ML-Agents Scene Block Diagram



For this research, each Drone Agent includes, (1) Agent Script, Handles OnActionReceived (processing joystick inputs), CollectObservations (monitoring states), and Heuristic (manual control for debugging); (2) Behavior Parameters: Defines the vector space for actions and observations. (3) Decision Requester: Controls the frequency of actions. By limiting the request rate, we can simulate more human-like reaction times, making the agent a better tutor for human pilots.

Proximal Policy Optimization (PPO) is an on-policy actor-critic algorithm that updates policies in small, clipped steps to ensure stability (OpenAI, 2017). PPO is particularly effective for drones learning to navigate complex environments (Yu et al., 2025).

The optimization objective is defined as Equation (16) shows.

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t[\min(r_t(\theta)\hat{A}_t, \text{textclip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t)] \tag{16}$$

- θ is the policy parameter
- $\hat{E}_t$ denotes the empirical expectation over timesteps
- r_t is the ratio of the probability under the new and old policies, respectively
- $\hat{A}_t$ is the estimated advantage at time t
- ε is a hyperparameter, usually 0.1 or 0.2

Compared with a PD AI agent, a reinforcement learning-based AI agent does not require repeated testing to fine-tune parameters. Decisions are executed through a Decision Requester, which makes it easier to adjust the request frequency and pacing. However, as a black-box approach, it can be more difficult to achieve optimal performance or interpret and refine its behavior.

Comparison of the XR Interaction Frameworks

Three widely used XR interaction frameworks are considered in this study: Unity XR Interaction Toolkit (XRI), Microsoft Mixed Reality Toolkit (MRTK), and Meta XR Interaction SDK.

Unity XR Interaction Toolkit (XRI) is a high-level, component-based interaction framework for developing virtual reality (VR) and augmented reality (AR) applications. It provides a unified system for handling 3D object manipulation and user interface interactions through Unity's input system.

Microsoft Mixed Reality Toolkit (MRTK) is an open-source, cross-platform framework designed for spatial interaction, originally targeting HoloLens devices but later extended to support VR platforms through OpenXR. MRTK offers advanced interaction capabilities, such as hand tracking, eye gaze interaction, and mixed reality user interface components (Dojcinovic

et al., 2026). While MRTK can be integrated with XRI, it functions as a standalone framework with its own interaction paradigms.

Meta XR Interaction SDK (formerly Oculus Integration / OVR XR) is a platform-specific framework optimized for Meta Quest devices. Unlike XRI and MRTK, it is tightly coupled with Meta's hardware and runtime, providing direct access to device-specific features such as head tracking, hand tracking, eye tracking, and facial expression tracking (T. Wang et al., 2026).

Having analyzed the existing development tools in the field of augmented reality, it is evident that not only the foundational game engine is important, but also the extensions that significantly speed up and simplify the development of such applications. Each solution has its own direction and is customized for certain hardware or operating systems. Therefore, many different extensions and the openness of some solutions allows them to be used not only for games and entertainment, but also in any areas of activity, including robotics, medicine, navigation, etc. (Shatokhin et al., 2025)

Previous research highlights that, beyond the choice of game engine, interaction frameworks play a critical role in simplifying development and accelerating prototyping in XR applications (Shatokhin et al., 2025). These frameworks differ in their level of abstraction, hardware dependency, and supported interaction modalities.

In this study, the interaction design is based on a floor-based XR reference coordinate system, where a handheld controller is used to emulate a radio transmitter. Given the need for rapid prototyping and flexible interaction design, MRTK is selected due to its rich interaction features and cross-platform capabilities.

Geographic Information Systems

AI-assisted environment generation

System Design Overview

Archetecture

Models

Environment Construction

Interaction and Control Design

Learning System Design

Data Collection Methods

Results

Control the Quadcopter in XR environment

The architecture of Yue's ultimate drone physics shown in Figure 9. A quad GameObject includes (1) Rigidbody, Unity's physics (2) YueInputModule, used to calculate controller axis. (3) YueDronePhysics, used to calculate the physics should applied on the drone's body with YueInputModule. (4) DroneXrEmulator, Map the Meta Quest Touch controller joystick data to YueInputModule.

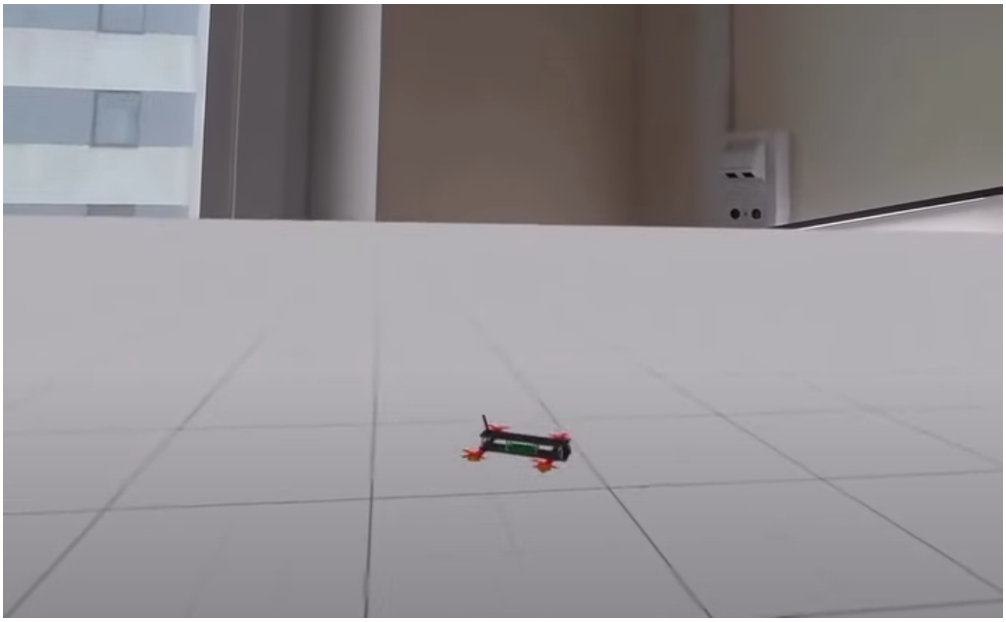
Figure 9
The Components of the Quad GameObject

GameObject : **Quad**
Rigidbody
YueInputModule
YueDronePhysics
DroneXrEmulator

To implement video passthrough, The scene should include the AR session into the scene, also add AR Raycast manager into the XR Origin. And add "AR Camera Manager" and "AR Camera background" on the MainCamera.

The result as shown in Figure 10

Figure 10
Controlling the drone physics in XR environment



Path Following with PID Controller

It is better to complete this section in a flat panel display. As this part is more algorithm centered. Instead of using the DroneXrEmulator in Figure 9, it is simpler to make another PidDroneEmulator which maps the calculated joystick value to YueInputModule.

To implement path following, two other components are included, PathFollower, cut the path into different targets for PidDroneEmulator to calculate errors and PathGenerator to build the path.

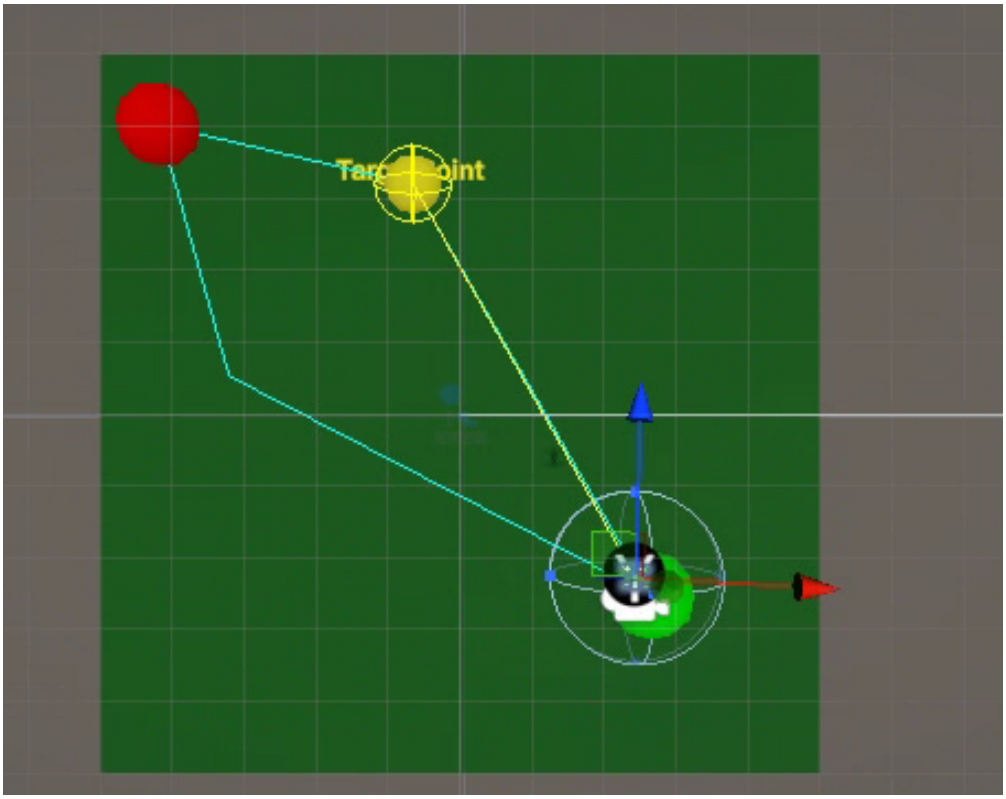
To move between points, the PathGenerator generates a path using two Cubic Bézier curves. Given four control points P_0, P_1, P_2, P_3, home to waypoint and back, the position $B(t)$ for $t \in [0,1]$ is defined as Equation (17).

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3 \tag{17}$$

The path is constructed relative to the Home and Target vectors, the drone will fly from Home and fly back, the path as shown in Figure 11. Path from Home to Waypoint using a

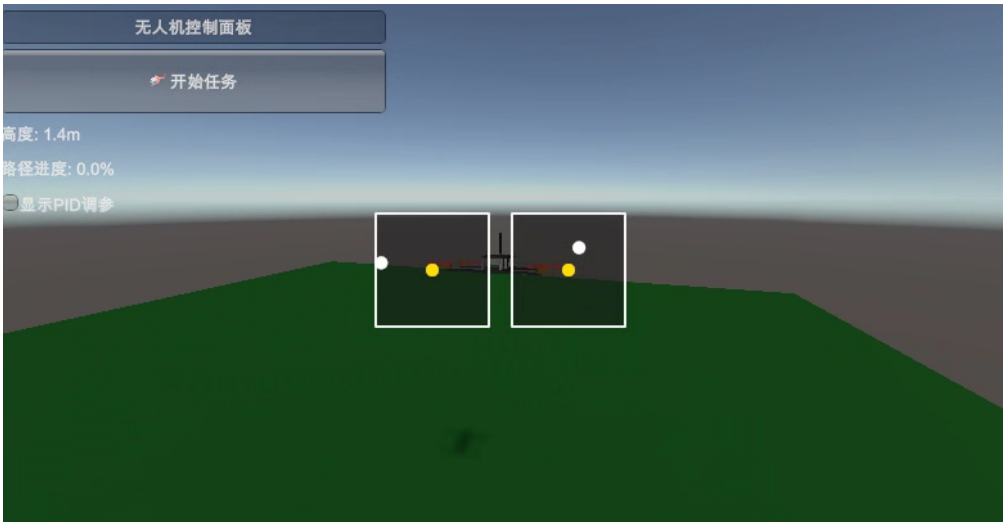
right-hand offset for control points; Path from Waypoint back to Home using a left-hand offset.

Figure 11
Path Design



By calculating these normalized joystick rates, the system can visualize the AI's calculation on the HUD.This serves as an AI collaboration hinter, showing human pilots the optimal stick positions for maintaining the designated path Figure 12.

Figure 12
hinter UI



Hovering with RL Agent

We implemented a 1D control mission: regulating the drone's vertical position (y) within a target band $[H-R, H+R]$, where H is the 'targetHeight' and R is the 'acceptableRadius'.

The following two values are being observed. (1) Normalized Height Error: $\text{clip}(\frac{y-H}{R}, -k, k)/k$ (where k is 'errorClampK'). This maps the error to a $[-1,1]$ range, preventing extreme spikes from destabilizing the gradient. (2) Vertical Velocity: v_y/v_{ref}, clipped to $[-1,1]$.

Rewards are calculated at each decision step (Δt): (1) Event-based: A firstArrivalReward is granted upon entering the target band for the first time per episode. (2) Dense/Steady-state: A livingRewardPerSecond is granted while maintaining position within the band. (3) Shaping: A penalty proportional to the normalized excess $(|e|-R)/R$ is applied when outside the band to discourage drifting.

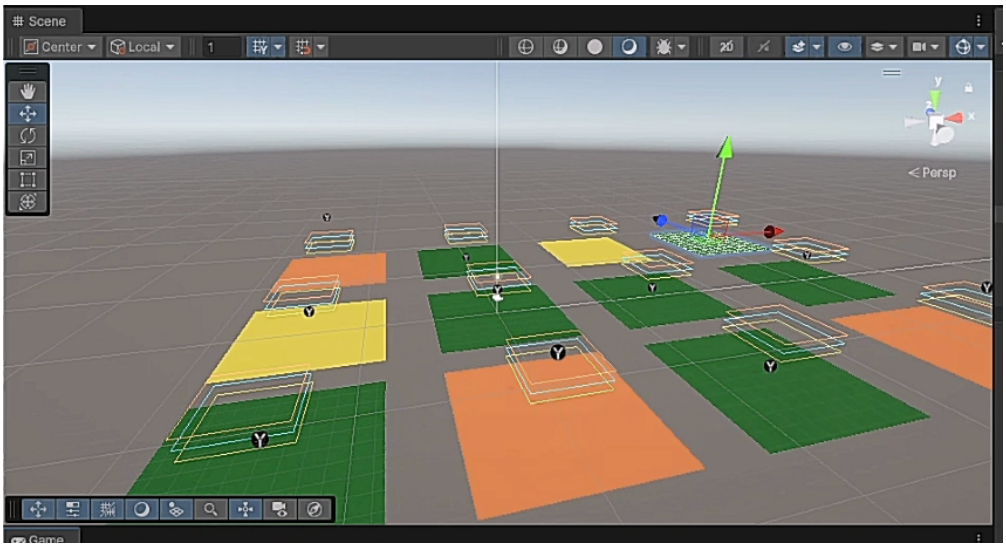
The agent was trained for 5×10^5 steps using the following parameters with Proximal Policy Optimization Equation (16) approach:

- Clipping parameter $\varepsilon = 0.2$
- GAE parameters $(\lambda, \gamma) = 0.99$
- Neural network: a 2-layer MLP with 128 hidden units

- Learning rate: 3×10^{-4} with linear decay

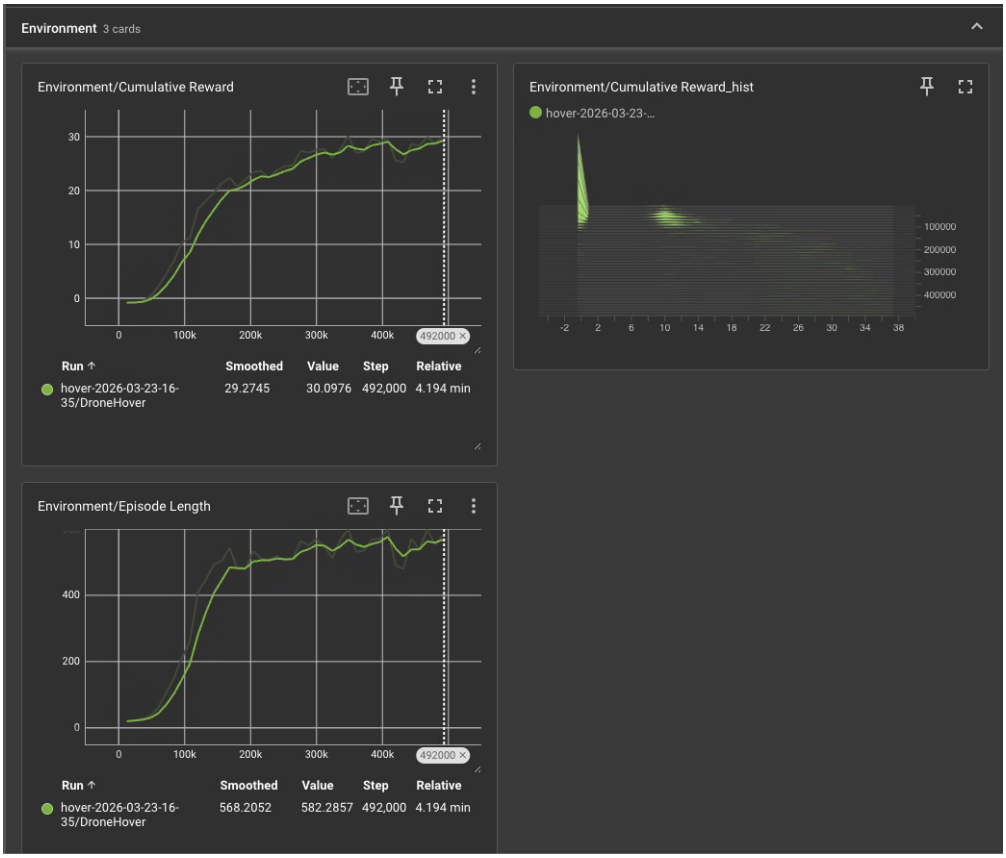
To monitor progress, we implemented a color-coded feedback system, Figure 13: Yellow: Below target; Orange: Above target; Blue: Stable hovering.

Figure 13
training process



From the train report, Cumulative Reward: Showed steady growth after 100k steps, plateauing at 400k, Figure 14. Episode Length: Positively correlated with rewards; as the agent improved, it survived longer without crashing (falling too low). Environment/Cumulative Reward Histograms: also shows when training time raises, reward start to raise.

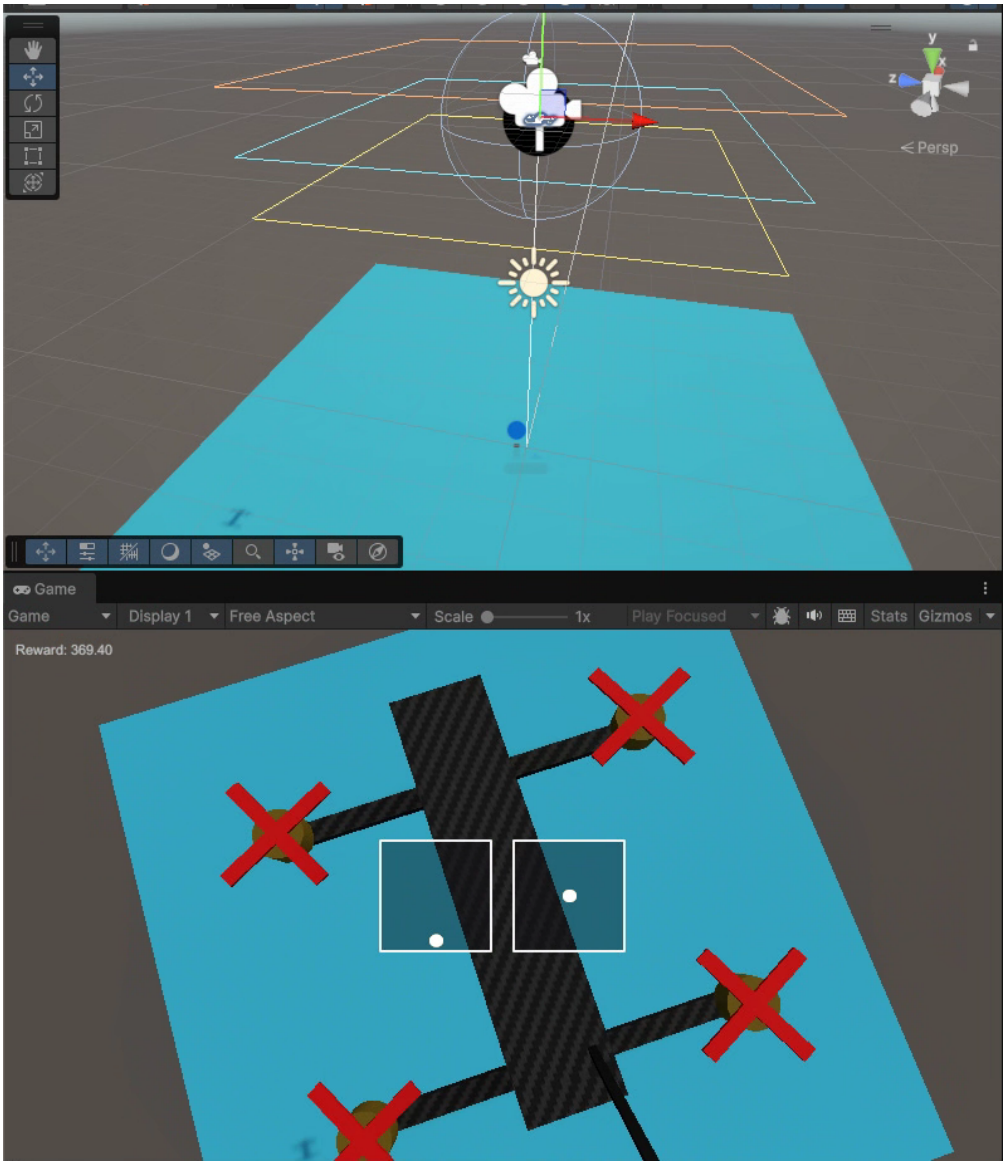
Figure 14
Training result of the RL agent



The RL agent successfully learned to hover with high precision. By adjusting the DecisionRequester, we created a smoother control hint for the UI, matching the pace of human perception Figure 15.

Figure 15

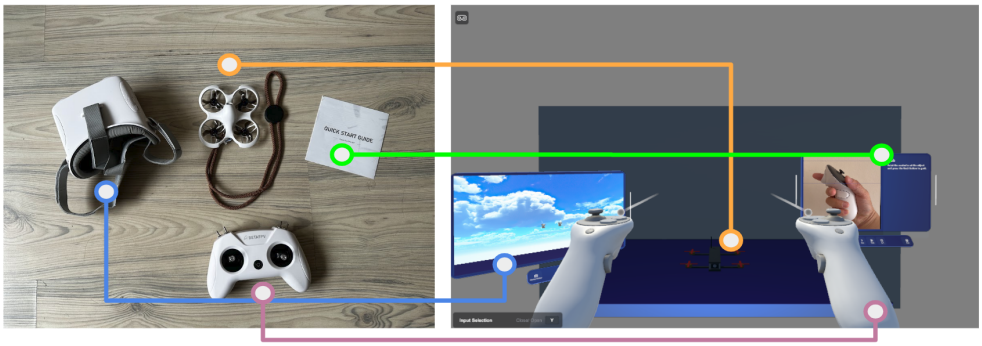
UI prototype of the RL learning agent



Tutorial on Game Interaction

Interaction Game Objects including the following elements. Quad, DroneVision and Instruct. The Quad GameObject act as the quadcopter. The droneVision GameObject uses Unity's RenderTexture to render the video feed from the quadcopter act as the FPV goggles. The instruct GameObject shows tutorials for player to learn simple joystick interaction of the quadcopter.

Figure 16
structure of the interaction elements



There are 6 interactions the player has to learn, 5 are inclusive of quadcopter's HCI Figure 3. Arm, use to start the rotors, is to drag the left stick down to zero. Throttle, increase or decrease the rotor speed, change the longitudinal force of the quadcopter's body, is to turn the left stick up and down. Yaw, turn the quadcopter's body left and right, is to turn the left stick left and down. Pitch, control the quadcopter lean forward and back, is to turn the right stick up and down. Roll, control the quadcopter lean left and right, is to turn the right stick left and right.

The last interaction is to Grip or Select, which is used to click on a button or drag some interactive element. It uses the grip button on either of the controllers.

Control the Quadcopter in VR environment

Drone Control Implementation

Physics and Simulation System

XR Interaction and UI Results

Discussion

RQ1: How Can FPV Quadcopter Flight Physics Be Simulated?

This study adopts Yue's Ultimate Drone physics model (Yuetility-Studios, 2022) as a reference implementation for simulating first-person view (FPV) quadcopter flight dynamics. The model employs a proportional-derivative (PD) control approach for flight stabilization, which is widely used in modern quadcopter systems.

In addition, a mapping strategy was developed to translate the controller profile of the Meta Quest into a simulated drone transmitter, thereby enabling intuitive user interaction within the virtual environment.

To ensure safe operation during simulation, the environment incorporates collider-based boundaries that constrain the quadcopter within a predefined area.

Comparison With Existing Simulators

FPV simulators (e.g., Liftoff, Uncrashed) primarily adopt cascaded proportional-integral-derivative (PID) control models to replicate real-world flight controllers. These systems typically support multiple controller configurations and provide interfaces for PID parameter tuning. However, such configurations can be difficult for beginners to set up with their own controllers.

Although most simulators provide a variety of selectable environments, they are primarily designed for flat-panel displays and do not offer the immersive learning enabled by extended reality (XR) environments.

Commercial FPV simulators commonly employ Acro (acrobat) mode to achieve realistic flight behavior. In contrast, this study focuses on a self-leveling flight mode, which constrains the drone's body angle and reduces the likelihood of crashes, thereby minimizing user frustration during early-stage learning.

Unlike existing simulators, the proposed system incorporates a human-AI collaboration mechanism that provides calculated control suggestions to support user decision-making.

Although Acro mode remains important for future investigation due to its higher realism,

it also introduces a greater likelihood of crashes. This raises several design considerations, including mechanisms for crash recovery, feedback systems for crash detection, and strategies for enabling AI agents to avoid collisions. From a game design perspective, it is also necessary to evaluate whether and how penalties should be applied following crashes.

RQ2: How can a human-AI collaboration agent be designed to assist users?

This study presents two approaches for designing a human-AI collaboration agent. The first approach is a transparent, white-box method based on proportional-integral-derivative (PID) control. The second approach is a black-box method using reinforcement learning (RL).

Comparison of PID and RL

To support user interaction and feedback, a heads-up display (HUD) was implemented to visualize the AI agent's real-time predictions of joystick inputs. The interface consists of two panels with crossed axes, where points represent the predicted joystick positions.

The results indicate that the RL-based solution provides greater flexibility during prototyping, as the frequency of agent decision-making can be adjusted dynamically. However, due to its black-box nature, effective fine-tuning remains challenging and represents an important direction for future research.

RQ3: What interaction methods enable high-fidelity XR prototyping?

This study employs the Mixed Reality Toolkit (MRTK) in combination with the Unity XR Toolkit to scaffold the user interface. Control inputs are implemented using the Meta Quest Touch Controller to emulate real-world quadcopter interaction. The user operates within a local-floor reference space, while a view-based heads-up display (HUD) provides real-time control status feedback.

To simulate first-person-view (FPV) goggles, Unity's RenderTexture is used to project the quadcopter's camera feed onto a virtual display panel. This design enables users to simultaneously perceive both the surrounding environment and the drone's onboard perspective, thereby supporting situational awareness and potentially reducing simulator sickness.

The system incorporates three interaction environments: (1) Additive Environment Blend,

an augmented reality setting in which users perform tutorial tasks within their physical surroundings; (2) a white-boxed playground, which serves as a controlled training environment for developing piloting skills; and (3) real city piloting, a virtual geographic environment that enables navigation within a simulated urban landscape. The design of these learning environments is grounded in experiential learning theory (Kolb, 1984), which conceptualizes learning as a cyclical process of experience, reflection, and experimentation.

When XR helps

This study positions XR as a spatial prototyping medium for FPV drone learning system design. XR enables direct manipulation of control mappings in a three-dimensional environment, reducing the need for abstract interface layout design and accelerating iterative testing of interaction strategies.

When XR is not preferable

When spatial arrangement is not part of the design problem, XR may introduce unnecessary complexity without improving simulation fidelity or development efficiency. In such cases, traditional simulation or parameter optimization approaches, such as PID tuning or reinforcement learning-based control refinement, are more efficient.

Limitations

Self-Leveling Flight Mode Only

For implementation simplicity, the current system is limited to self-leveling flight dynamics. Although Yue's Ultimate Drone physics model supports both self-leveling and acrobatic modes, this study focuses exclusively on the former to align with the design of the human-AI collaboration agents, which are optimized for stable flight control.

Consequently, acrobatic behaviors are not supported in the current implementation, which restricts the system's applicability to advanced flight scenarios.

Visual Feedback Only

Extended reality (XR) systems can support multimodal feedback. Additional modalities, such as haptic feedback and spatial audio, were considered in the design phase of the human-AI collaboration system, but they were not implemented in the current system.

This decision was driven by the additional design complexity associated with determining appropriate feedback types, intensity levels, and integration strategies. These aspects remain open research challenges for future work.

No Participant Testing

This study primarily focuses on system design and implementation rather than controlled user evaluation. As XR-based control interfaces are still relatively unfamiliar to many users, variability in prior experience may significantly influence task performance.

This makes it difficult to isolate learning effects from interface unfamiliarity, thereby limiting the interpretability of the user study results.

Future Work

Evaluation Methodology for XR-Based Drone Prototyping

Future work should develop more reliable evaluation methods for XR-based drone prototyping systems. In current user studies, participants may have very different levels of experience with XR devices, which can strongly affect their performance and learning speed.

To address this issue, future studies should consider designing evaluation protocols that reduce the influence of prior XR familiarity.

Extending to Acrobatic Flight Mode

Future work should extend the current self-leveling flight system to include acrobatic (acro) mode. While acro mode enables more expressive and realistic quadcopter control, it also introduces significantly higher control complexity and instability. This extension raises several research challenges, including crash detection, recovery mechanisms, and AI-assisted strategies.

From a game design perspective, it is also important to investigate circumstances, such as

penalties or reset mechanisms.

Designing Multimodal Feedback Mechanisms

Future research should explore the integration of multimodal feedback, including audio, and haptic channels, to enhance user awareness during flight control. Rather than simply adding additional feedback modalities, it is important to investigate their placement, timing, and intensity, as these parameters may significantly affect cognitive load and learning performance. A systematic study of feedback design space could provide guidelines for more effective XR interaction design.

Reinforcement Learning for Path-Following Control

While hovering control can be effectively modeled using distance-based feedback signals, path-following tasks require reasoning over multiple spatial and temporal variables. Manually designing these features can become inefficient and non-scalable.

Future work should explore reinforcement learning approaches to automatically learn control policies from richer state representations. This may enable a transition from target-based navigation to more general vision-based flight control systems.

Imitation Learning for Human-Guided Control

Although imitation learning provides a promising direction for human-aligned control policies, it requires large-scale and high-quality human demonstration data. In the context of XR drone interaction, collecting such data introduces challenges in consistency, labeling, and user variability.

Future research should investigate efficient data collection strategies, including semi-automated annotation or human-in-the-loop learning, to reduce the overhead of dataset construction and improve training efficiency.

Summary

Goals

Develop XR training system with AI guidance.

Results

Functional system with immersive interaction.

Generalization

XR supports skill training.

Practical Use

Prototype training framework.

Future Study

Learning validation and system expansion.

References

Almeida, P., & Mendes, R. (2010). Learning style preferences across disciplines. *The International Journal of Diversity in Organizations, Communities, and Nations: Annual Review*, *10*(2), 285–302. https://doi.org/10.18848/1447-9532/CGP/v10i02/39847

Bailenson, J. N., Beams, B., Brown, J., et al. (2024). Seeing the world through digital prisms. *Technology, Mind, and Behavior*, *5*(2), 43–58. https://doi.org/10.1037/tmb0000129

Barnett, S. M., & Ceci, S. J. (2002). When and where do we apply what we learn?: A taxonomy for far transfer. *Psychological Bulletin*, *128*(4), 612–637. https://doi.org/10.1037/0033-2909.128.4.612

Barto, A. G. (2019). Reinforcement learning: Connections, surprises, challenges. *AI Magazine*, *40*(1), 3–15. https://doi.org/10.1609/aimag.v40i1.2844

Beck, K., & Andres, C. (2004). *Extreme programming explained: Embrace change* (2nd). Addison-Wesley Professional.

Bennett, S. (1993). Development of the pid controller. *IEEE Control Systems*, *13*(6), 58–62. https://doi.org/10.1109/37.248006

BetaFPV. (2022). *Literadio 3 radio transmitter user manual*. https://support.betafpv.com/hc/en-us/article_attachments/55907267618073

Billinghurst, M., Clark, A., & Lee, G. (2015). A survey of augmented reality. *Foundations and Trends® in Human-Computer Interaction*, *8*, 73–272. https://doi.org/10.1561/1100000049

Bowman, D. A., & Hodges, L. F. (1997). An evaluation of techniques for grabbing and manipulating remote objects in immersive virtual environments. *Proceedings of the 1997 symposium on Interactive 3D graphics - SI3D '97*, 35–ff. https://doi.org/10.1145/253284.253301

Dablander, M. (2024, December 18). *Future research avenues for artificial intelligence in digital gaming: An exploratory report* [arXiv.org]. Retrieved April 30, 2026, from https://arxiv.org/abs/2412.14085v1

Dai, C.-P., & Ke, F. (2022). Educational applications of artificial intelligence in simulation-based learning: A systematic mapping review. *Computers and Education: Artificial Intelligence*, *3*, 100087. https://doi.org/10.1016/j.caeai.2022.100087

Davari, S., & Bowman, D. A. (2024, November 4). Towards context-aware adaptation in extended reality: A design space for XR interfaces and an adaptive placement strategy. https://doi.org/10.48550/arXiv.2411.02607

Dojcinovic, S., Strecker-Bischoff, J., Mayer, S., & Bektaş, K. (2026). Personalized recommendations in mixed reality enhance explanation satisfaction and hedonic user experience in board game learning. *Proceedings of the 31st International Conference on Intelligent User Interfaces*, 1263–1282. https://doi.org/10.1145/3742413.3789129

Dutescu, R., Apostol, M., Mocanu, I., & Barbu, M. (2025). Virtual reality in training: A multidimensional analysis of technologies, challenges and paradigms. *Proceedings of the International Conference on Virtual Learning - VIRTUAL LEARNING - VIRTUAL REALITY (20th edition)*, 129–142. https://doi.org/10.58503/icvl-v20y202511

Ebeid, E., Skriver, M., Terkildsen, K. H., Jensen, K., & Schultz, U. P. (2018). A survey of open-source UAV flight controllers and flight simulators. *Microprocessors and Microsystems*, *61*, 11–20. https://doi.org/10.1016/j.micpro.2018.05.002

Ezhil, S. (2025). A comparative analysis of unity, unreal engine, godot, and cryengine: Technical features, licensing models, and suitability for modern game development. *Journal of Game Theory*. https://doi.org/10.5281/zenodo.17383448

Game Developers Conference. (2026). 2026 state of the game industry report. https://reg.gdconf.com/2026-SOTI

Gao, Q., Xu, W., Pan, H., Shen, M., & Gao, Z. (2025). Human-centered human-ai collaboration (hchac). https://arxiv.org/abs/2505.22477

García-Sánchez, P. (2019). Georgios n. yannakakis and julian togelius: Artificial intelligence and games: Springer, 2018, print ISBN: 978-3-319-63518-7, online ISBN:

978-3-319-63519-4, https://doi.org/10.1007/978-3-319-63519-4. *Genetic Programming and Evolvable Machines*, *20*(1), 143–145. https://doi.org/10.1007/s10710-018-9337-0

Ghazbi, S. N., Aghli, Y., Alimohammadi, M., & Akbari, A. A. (2016). Quadrotors unmanned aerial vehicles: A review. *International Journal on Smart Sensing and Intelligent Systems*, *9*(1), 309–333. https://doi.org/10.21307/ijssis-2017-872

Guide, F. D. (2026, January 19). *Best FPV simulators 2026 | liftoff vs velocidrone* [FPV drone guide]. Retrieved April 30, 2026, from https://fpvdroneguide.com/blog/best-fpv-simulators-2026

Idrissi, M., Salami, M., & Annaz, F. (2022). A review of quadrotor unmanned aerial vehicles: Applications, architectural design and control algorithms. *Journal of Intelligent & Robotic Systems*, *104*(2), 22. https://doi.org/10.1007/s10846-021-01527-7

Jong, M. S. Y., Dong, A., & Luk, E. (2017). Design-based research on teacher facilitation practices for serious gaming in formal schooling. *Research and Practice in Technology Enhanced Learning*, *12*(1), 19. https://doi.org/10.1186/s41039-017-0056-6

Kaelbling, L. P., Littman, M. L., Moore, A. W., & Hall, S. (1996). Reinforcement learning: A survey. *Journal of Artificial Intelligence Research*. https://doi.org/10.1613/jair.301

Karahan, M. (2025). Development stages of quadrotors from past to present: A review. *Drones*, *9*, 840. https://doi.org/10.3390/drones9120840

Keith, C. (2010). *Agile game development with scrum (addison-wesley signature)* (1st). Addison-Wesley Professional.

Khronos OpenXR Working Group. (2024). Openxr 1.1 specification. https://registry.khronos.org/OpenXR/specs/1.1/html/xrspec.html

Knobovitch, R. M., Tokuno, J., Botelho, F., Fried, H. B., Carver, T. E., & Fried, G. M. (2025). Virtual reality training improves procedural skills in mannequin-based simulation in medical students: A pilot randomized controlled trial. *Surgical Innovation*, *32*(4), 364–373. https://doi.org/10.1177/15533506251334693

Kolb, D. A. (1984). *Experiential learning: Experience as the source of learning and development* [Accessed: 2006-05-31]. Prentice Hall. http://www.learningfromexperience.com/images/uploads/process-of-experiential-learning.pdf

Lin, Z., Ehsan, U., Agarwal, R., Dani, S., Vashishth, V., & Riedl, M. (2023, May 3). *Beyond prompts: Exploring the design space of mixed-initiative co-creativity systems* [arXiv.org]. Retrieved April 30, 2026, from https://arxiv.org/abs/2305.07465v1

Liu, C., Meng, S., Zheng, W., & Zhou, Z. (2025). Research on the impact of immersive virtual reality classroom on student experience and concentration. *Virtual Reality*, *29*(2), 82. https://doi.org/10.1007/s10055-025-01153-w

Lovesay, E. J. (Ed.). (2003). *Contemporary ergonomics 1993*. CRC Press.

Luukkonen, T. (2011). Modelling and control of quadcopter.

Machado, T., Fassbender, D., Taheri, A., Eriksson, D., Gupta, H., Molaei, A., Forte, P., Rai, P. K., Ghabcheloo, R., Makinen, S., Lilienthal, A. J., Andreasson, H., & Geimer, M. (2021). Autonomous heavy-duty mobile machinery: A multidisciplinary collaborative challenge. *2021 IEEE International Conference on Technology and Entrepreneurship (ICTE)*, 1–8. https://doi.org/10.1109/ICTE51655.2021.9584498

Marougkas, A., Troussas, C., Krouska, A., & Sgouropoulou, C. (2023). Virtual reality in education: A review of learning theories, approaches and methodologies for the last decade. *Electronics*, *12*(13), 2832. https://doi.org/10.3390/electronics12132832

Mine, M. R. (1995). Virtual environment interaction techniques.

Narayanasamy, V., Wong, K. W., Fung, C. C., & Rai, S. (2006). Distinguishing games and simulation games from simulators. *Computers in Entertainment*, *4*(2), 9. https://doi.org/10.1145/1129006.1129021

OpenAI. (2017). Proximal policy optimization. https://openai.com/index/openai-baselines-ppo/

Otto, A., Agatz, N., Campbell, J., Golden, B., & Pesch, E. (2018). Optimization approaches for civil applications of unmanned aerial vehicles (UAVs) or aerial drones: A survey. *Networks*, *72*(4), 411–458. https://doi.org/10.1002/net.21818

Pappas, G., Lappas, D., Karampelas, P., Siegel, J. E., Templalexis, I., Stavrou, S., & Krishnan, S. (2025). Generic drone simulator: Design, development and user testing in a virtual search and rescue mission. *IEEE Computer Graphics and Applications*, 1–14. https://doi.org/10.1109/MCG.2025.3560537

Paredes, J., Sharma, P., Ha, B., et al. (2021). Development, implementation, and experimental outdoor evaluation of quadcopter controllers for computationally limited embedded systems. *arXiv preprint arXiv:2105.14231*. https://doi.org/10.48550/arXiv.2105.14231

Rendevski, N., Trajcevska, D., Dimovski, M., et al. (2022). Pc vr vs standalone vr fully-immersive applications: History, technical aspects and performance. *ICEST*, 1–4. https://doi.org/10.1109/ICEST55168.2022.9828656

Rolland, J., & Hua, H. (2005). Head-mounted display systems. *Encyclopedia of Optical Engineering*.

Schmalstieg, D., & Höllerer, T. (2016, June 10). *Augmented reality - principles and practice*.

Shah, S., Dey, D., Lovett, C., & Kapoor, A. (2017, July 18). AirSim: High-fidelity visual and physical simulation for autonomous vehicles. https://doi.org/10.48550/arXiv.1705.05065

Shatokhin, O., Dzedzickis, A., Pečiulienė, M., & Bučinskas, V. (2025). Extended reality: Types and applications. *Applied Sciences*, *15*, 3282. https://doi.org/10.3390/app15063282

Usama, A., Dora, M., Elhadidy, M., Khater, H., & Alkelany, O. (2021). First person view drone - fpv. *The International Undergraduate Research Conference*, *5*(5), 437–440. https://journals.ekb.eg/article_246400.html

Waldow, K., Kleinbeck, C., Fuhrmann, A., & Roth, D. (2025). Investigating the impact of video pass-through embodiment on presence and performance in virtual reality. *IEEE Transactions on Visualization and Computer Graphics*, *31*(5), 2364–2373. https://doi.org/10.1109/TVCG.2025.3549891

Wang, H., Lai, X., Huang, S., Dai, X., Zhao, X., & Wang, Y. (2026). The influence of prior perception, attitude, and immediate knowledge of AI on adolescents' preferences for high-

and low-replaceable jobs. *Behavioral Sciences*, *16*(1), 72. https://doi.org/10.3390/bs16010072

Wang, T., Byeon, J., Yehia, A., Xu, Y., Park, J., Zeng, T., Chen, S., Wang, Z., Jiao, J., & Claudel, C. (2026, March 6). XR-DT: Extended reality-enhanced digital twin for safe motion planning via human-aware model predictive path integral control. https://doi.org/10.48550/arXiv.2512.05270

World Wide Web Consortium. (2022). Webxr device api. https://www.w3.org/TR/webxr/

Yu, H., De Wagter, C., & De Croon, G. C. H. E. (2025). Depth transfer: Learning to see like a simulator for real-world drone navigation. *IEEE Robotics and Automation Letters*, *10*(11), 11848–11855. https://doi.org/10.1109/LRA.2025.3617729

Yuetility-Studios. (2022). Yue ultimate fpv drone physics (version 1.0) [computer software]. https://assetstore.unity.com/packages/tools/physics/yue-ultimate-fpv-drone-physics-231651

Zhang, B., Xu, M., & Pan, Z. (2025, January 20). *Human-AI collaborative game testing with vision language models* [arXiv.org]. Retrieved April 30, 2026, from https://arxiv.org/abs/2501.11782v2

Appendix